

Skill 3

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <termios.h>
```

```
#define MAX_HISTORY 10
```

```
#define MAX_COMMAND 100
```

```
char history[MAX_HISTORY][MAX_COMMAND];
```

```
int history_count = 0;
```

```
int current_history = 0;
```

```
struct termios original_terminal;
```

```
/* Restore terminal settings */
```

```
void restore_terminal()
```

```
{
```

```
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &original_terminal);
```

```
}
```

```
/* Enable raw mode */
```

```
void enable_raw_mode()
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, &original_terminal);

    raw = original_terminal;

    raw.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}
```

/* Store command in history */

```
void store_command(char *command)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);

        history_count++;
    }
    else
    {
        /* Shift old commands */
        for (int i = 0; i < MAX_HISTORY - 1; i++)
        {
            strcpy(history[i], history[i + 1]);
```

```
}
```

```
strcpy(history[MAX_HISTORY - 1], command);
```

```
}
```

```
current_history = history_count;
```

```
}
```

```
/* Clear current input from screen */
```

```
void clear_input(char *buffer)
```

```
{
```

```
    int length = strlen(buffer);
```

```
    for (int i = 0; i < length; i++)
```

```
    {
```

```
        printf("\b \b");
```

```
    }
```

```
}
```

```
/* Replace input buffer */
```

```
void update_buffer(char *buffer, char *command)
```

```
{
```

```
    clear_input(buffer);
```

```
    strcpy(buffer, command);
```

```
    printf("%s", buffer);
```

```
    fflush(stdout);
}

int main()
{
    char buffer[MAX_COMMAND];

    int index = 0;

    char c;

    enable_raw_mode();

    printf("Mini Command History Program\n");
    printf("Type commands.\n");
    printf("Use UP/DOWN arrows for history.\n");
    printf("Type exit to quit.\n\n");

    printf("> ");

    while (1)
    {
        c = getchar();

        /* ENTER */
        if (c == '\n' || c == '\r')
        {
            buffer[index] = '\0';
```

```
printf("\n");
```

```
if (strcmp(buffer, "exit") == 0)
```

```
{
```

```
    break;
```

```
}
```

```
if (strlen(buffer) > 0)
```

```
{
```

```
    store_command(buffer);
```

```
    printf("Command stored: %s\n", buffer);
```

```
}
```

```
index = 0;
```

```
buffer[0] = '\0';
```

```
current_history = history_count;
```

```
printf("> ");
```

```
}
```

```
/* BACKSPACE */
```

```
else if (c == 127 || c == '\b')
```

```
{
```

```
    if (index > 0)
```

```

{
    index--;

    buffer[index] = '\0';

    printf("\b\b");

    fflush(stdout);
}

}

/* ESCAPE SEQUENCE */
else if (c == '\033')
{
    char seq1 = getchar();
    char seq2 = getchar();

    if (seq1 == '[')
    {
        /* UP ARROW */
        if (seq2 == 'A')
        {
            if (current_history > 0)
            {
                current_history--;

                update_buffer(
                    buffer,

```

```

        history[current_history]
    );

    index = strlen(buffer);
}
}

/* DOWN ARROW */
else if (seq2 == 'B')
{
    if (current_history < history_count - 1)
    {
        current_history++;

        update_buffer(
            buffer,
            history[current_history]
        );

        index = strlen(buffer);
    }
    else
    {
        clear_input(buffer);

        buffer[0] = '\0';

        index = 0;
    }
}

```

```

        current_history = history_count;
    }
}
}
}

/* NORMAL CHARACTER */
else
{
    if (index < MAX_COMMAND - 1)
    {
        buffer[index] = c;

        index++;

        buffer[index] = '\0';
        printf("%c", c);
        fflush(stdout);
    }
}

restore_terminal();
printf("\nTerminal restored.\n");

return 0;
}

```

```

snigdha@Snigdha:~/practical6$ mkdir skill3
snigdha@Snigdha:~/practical6$ cd skill3

snigdha@Snigdha:~/practical6/skill3$
snigdha@Snigdha:~/practical6/skill3$ nano history.c
snigdha@Snigdha:~/practical6/skill3$ gcc history.c -o history

snigdha@Snigdha:~/practical6/skill3$
snigdha@Snigdha:~/practical6/skill3$ ./history
Mini Command History Program
Type commands.
Use UP/DOWN arrows for history.
Type exit to quit.

> hello
Command stored: hello
> pwd
Command stored: pwd
> date
Command stored: date
> exit

Terminal restored.

```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
typedef struct Node
```

```
{
```

```
    char *data;
```

```
    struct Node *next;
```

```
} Node;
```

```
/* Create a new linked list node */
```

```
Node *create_node(const char *text)
```

```
{  
    Node *new_node;  
  
    new_node = malloc(sizeof(Node));  
  
    if (new_node == NULL)  
    {  
        printf("Memory allocation failed.\n");  
  
        exit(1);  
    }  
  
    /* Dynamically allocate buffer */  
    new_node->data = malloc(strlen(text) + 1);  
  
    if (new_node->data == NULL)  
    {  
        printf("Memory allocation failed.\n");  
  
        free(new_node);  
  
        exit(1);  
    }  
  
    strcpy(new_node->data, text);  
  
    new_node->next = NULL;
```

```
    return new_node;
}

/* Add node to linked list */
void add_node(Node **head, const char *text)
{
    Node *new_node = create_node(text);

    if (*head == NULL)
    {
        *head = new_node;
    }
    else
    {
        Node *temp = *head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = new_node;
    }
}
```

```
/* Print linked list */
```

```
void print_list(Node *head)
{
    printf("\nLinked List Contents:\n");

    while (head != NULL)
    {
        printf("%s\n", head->data);

        head = head->next;
    }
}
```

```
/* Release all memory */
void free_list(Node *head)
{
    Node *temp;

    while (head != NULL)
    {
        temp = head;

        head = head->next;

        free(temp->data);

        free(temp);
    }
}
```

```
}
```

```
int main()
```

```
{
```

```
    int size = 2;
```

```
    int count = 0;
```

```
    char **commands;
```

```
    Node *head = NULL;
```

```
    /* Dynamically allocate array */
```

```
    commands = malloc(size * sizeof(char *));
```

```
    if (commands == NULL)
```

```
    {
```

```
        printf("Memory allocation failed.\n");
```

```
        return 1;
```

```
    }
```

```
    printf("Dynamic Array and Linked List Demo\n");
```

```
    /* Add commands */
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
char input[100];

printf("Enter command %d: ", i + 1);

fgets(input, sizeof(input), stdin);

input[strcspn(input, "\n")] = '\0';

/* Resize array when full */
if (count == size)
{
    size = size * 2;

    char **temp;

    temp = realloc(
        commands,
        size * sizeof(char *)
    );

    if (temp == NULL)
    {
        printf("Memory resize failed.\n");

        break;
    }

    commands = temp;
```

```

        printf("Array resized to %d\n", size);
    }

    /* Dynamically allocate string */
    commands[count] =
        malloc(strlen(input) + 1);

    if (commands[count] == NULL)
    {
        printf("Memory allocation failed.\n");

        break;
    }

    strcpy(commands[count], input);
    /* Add to linked list */
    add_node(&head, input);
    count++;
}

printf("\nDynamic Array Contents:\n");
for (int i = 0; i < count; i++)
{
    printf("%s\n", commands[i]);
}

print_list(head);

/* Free dynamic array memory */
for (int i = 0; i < count; i++)

```

```

{
    free(commands[i]);
}

free(commands);

/* Free linked list */

free_list(head);

printf("\nAll memory released correctly.\n");

return 0;
}

```

```

snigdha@Snigdha:~/practical6/skill3$ nano memory.c
snigdha@Snigdha:~/practical6/skill3$ gcc memory.c -o memory

snigdha@Snigdha:~/practical6/skill3$
snigdha@Snigdha:~/practical6/skill3$ ./memory
Dynamic Array and Linked List Demo
Enter command 1: ls
Enter command 2: ped
Enter command 3: date
Array resized to 4
Enter command 4: whoami
Enter command 5: history
Array resized to 8

Dynamic Array Contents:
ls
ped
date
whoami
history

Linked List Contents:
ls
ped
date
whoami
history

All memory released correctly.
snigdha@Snigdha:~/practical6/skill3$

```